

SPOONS

by FiveMinds

EF Core Code — Individual Task Submission

Restaurant Ordering & Table Reservation · Codeline Spark to Code 2026

1. Assignment

Assigned models: Reservation and OrderItem.

Scope of this task: write the EF Core entity classes for these two models and their Fluent API configuration (mapping, relationships, delete behavior, indexes). These files are submitted on their own — they plug into the team's shared ApplicationDbContext once merged; they do not modify anything outside the two assigned models.

2. Reservation

A customer's booking of a table for a given date, time, and party size.

Field	Type	Notes
Id	int (PK)	Identity column.
UserId	int (FK)	Required. References User — the customer who booked.
TableId	int (FK)	Required. References Table — the table being reserved.
ReservationDate	DateOnly	Calendar date of the booking.
ReservationTime	TimeOnly	Time of the booking.
PartySize	int	Number of guests.
Status	ReservationStatus (enum)	Pending → Confirmed → Completed, or Cancelled. Defined in the shared Enums.cs.
CreatedAt	DateTime	UTC timestamp, set on insert.

Relationships

- Reservation N — 1 User (many-to-one, required): one customer can have many reservations.
- Reservation N — 1 Table (many-to-one, required): one table can have many reservations over time.

Design decisions

- Both FKs are required — a reservation cannot exist without a customer and a table.
- DeleteBehavior.Restrict on both relationships: a User or Table with existing reservations cannot be deleted outright. This protects booking history and forces an explicit cancel/reassign step first, instead of silently cascading deletes into a customer's reservation record.
- Composite index on (TableId, ReservationDate, ReservationTime) to make the 'is this table free at this date/time' availability check and the per-table upcoming-reservations lookup efficient.

3. OrderItem

A single line item within an Order — one menu item plus a quantity.

Field	Type	Notes
Id	int (PK)	Identity column.
OrderId	int (FK)	Required. References Order — the parent order.
MenuItemId	int (FK)	Required. References MenuItem — the item ordered.
Quantity	int	Units ordered.
UnitPrice	decimal(10,2)	Snapshot of MenuItem.Price at order time.
Subtotal	decimal(10,2)	UnitPrice × Quantity, stored (not computed on read).

Relationships

- OrderItem N — 1 Order (many-to-one, required): one order can have many line items.
- OrderItem N — 1 MenuItem (many-to-one, required): one menu item can appear on many order lines.

Design decisions

- UnitPrice is captured at order time rather than read live from MenuItem.Price, so a later menu price change never retroactively changes historical order totals.
- DeleteBehavior.Cascade from Order → OrderItem: a line item has no meaning outside its parent order, so deleting an order deletes its lines with it.
- DeleteBehavior.Restrict from MenuItem → OrderItem: a menu item that has ever been ordered cannot be hard-deleted, since that would corrupt order history. It should be deactivated (IsAvailable = false) instead of removed.
- Separate indexes on OrderId and MenuItemId support the order-detail view (all lines for an order) and reporting queries (how many times an item has been ordered).

4. Files in this submission

Models/Reservation.cs

Models/OrderItem.cs

Data/Configurations/ReservationConfiguration.cs

Data/Configurations/OrderItemConfiguration.cs

Both configuration classes implement `IEntityTypeConfiguration<T>` rather than being added inline to the shared `ApplicationDbContext.OnModelCreating` method. This was a deliberate choice for team workflow: with several people assigned different models in the same `DbContext` file, inline Fluent API blocks are a common source of merge conflicts. A separate config class per model can be added, reviewed, and merged independently.

5. Integration instructions (for whoever merges this in)

1. Copy `Models/Reservation.cs` and `Models/OrderItem.cs` into `RestaurantApi/Models/` (they assume `User`, `Table`, `Order`, `MenuItem`, and the `ReservationStatus` enum already exist there, as per the shared ERD).

2. Copy `Data/Configurations/ReservationConfiguration.cs` and `OrderItemConfiguration.cs` into `RestaurantApi/Data/Configurations/`.

3. In `ApplicationDbContext.OnModelCreating`, either:

```
modelBuilder.ApplyConfiguration(new ReservationConfiguration());
```

```
modelBuilder.ApplyConfiguration(new OrderItemConfiguration());
```

— or, to pick up every configuration class in the project automatically:

```
modelBuilder.ApplyConfigurationsFromAssembly(typeof(Program).Assembly);
```

4. Ensure `DbSet<Reservation>` and `DbSet<OrderItem>` are exposed on `ApplicationDbContext` (already present in the shared context).

5. Run a migration (`dotnet ef migrations add ...`) once merged with the rest of the team's models.

Prepared for team review before Mentor approval and Slack distribution of the final Restaurant_ERD_and_Data_Mapping. — Spoons by FiveMinds